

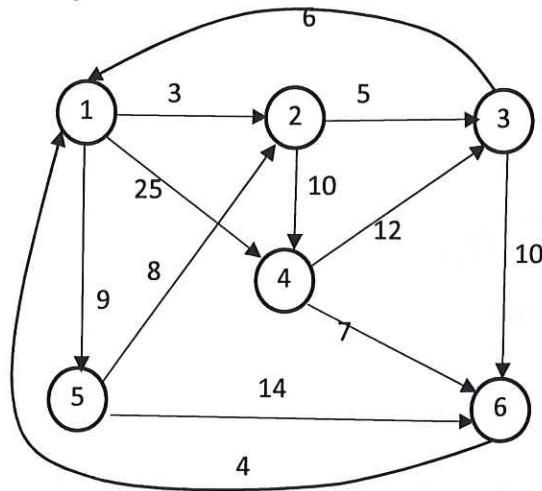
BITS PILANI, DUBAI CAMPUS
Dubai International Academic City, Dubai.
Third Year - II Semester 2017-18
Design and Analysis of Algorithms (CS F364)
Test -2 (Open Book)

Date: 11.04.2018 Max Marks: 40

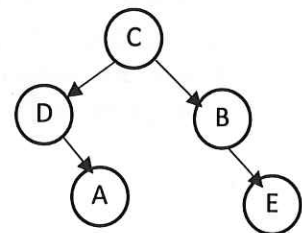
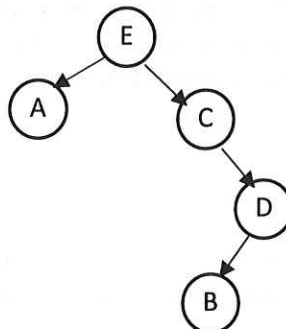
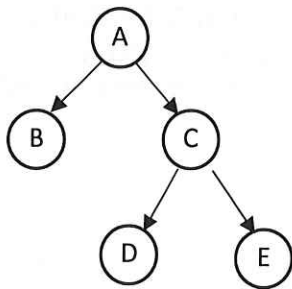
Time: 8.30 to 9.20 am, 50 minutes

Weightage: 20%

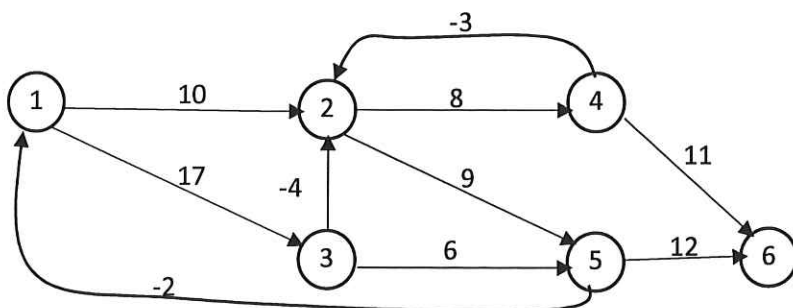
- For given graph, determine all pair shortest path. Write the adjacency matrix for the given graph. Provide the computation of cost matrix at each iteration along with showing the every steps to calculate it. [8 marks]



- Consider 5 symbols {A, B, C, D, E}, we have given the following search tree for these 5 symbols. Compute the cost of each tree. Assume that each symbol is searched with equal probability. [8 marks]



- For given following graph, determine the matrix for single source shortest path (single source all destination). Consider node 1 as the source and compute the matrix representing the single source all destination shortest path. [8 marks]



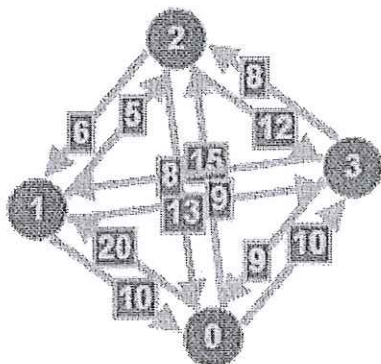
Q.4. Consider the following adjacency matrix of a graph:

```
[0 20 15 10]
[10 0 5 9]
[13 6 0 12]
[9 8 8 0]
```

Draw the graph corresponding to this adjacency matrix and compute the Travelling Salesman Problem optimal tour of this graph. Display the optimal tour and mention it's length.

[6 marks]

Solution:



TSP Tour: 0->3->2->1->0 (as per above graph) $(10 + 8 + 6 + 10) = 34$

$F(0: 1,2,3) = 10 + F(3: 2,1) = 10 + 8 + F(2: 1) = 10 + 8 + 6 + 10 = 34.$

Q.5. Justify why $P=NP?$ is a million dollar problem (Prize offered by Clay Mathematical Institute). Explain the problem in detail and describe it's importance in today's information era. Also mention four algorithmic problems which are known to be NP-complete. [4 marks]

Solution:

Digital cryptography rests on the difficulty of factoring numbers. Hence if $P=NP$ we would have failure of Cryptography which rests on high computational complexity.

Q.6. Give a non-deterministic algorithm for factoring a number (assume that the range of possible factors is finite (say 2 to some arbitrary whole number m)). Derive it's time complexity. Would the non-deterministic algorithm still work if the range of possible factors is not finite but infinite? [6 marks]

Solution:

1. Guess the factors of the number.

2. Multiply them in polynomial time and verify product equal to the original number.

BITS PILANI, DUBAI CAMPUS
Dubai International Academic City, Dubai.
Third Year - II Semester 2017-18
Design and Analysis of Algorithms (CS F364)
Test -1 (Closed Book)

Date: 27.02.2018

Max Marks: 40

Time: 8.30 to 9.20 am, 50 minutes

Weightage: 20%

1. Write an algorithm for Binary search with the following all the requirements
 - a. The array elements are sorted descending order
 - b. When element is present in the array, return its position in the array
 - c. When element is not present in the array, return its appropriate index such that array remains sorted in descending order

```
2. int beg=0,end=ub,key,mid=(beg+end)/2,loc=-1;
3.     printf("\n enter the element you wanna search:");
4.     scanf("%d",&key);
5.     while((beg<=end)&&(key!=la[mid]))
6.     {
7.         mid=(beg+end)/2;
8.         if(key==la[mid])
9.         {
10.            printf("\n search is sucessful");
11.            loc=mid;
12.            printf("\n element possion is %d in array",loc+1);
13.        }
14.        else
15.        {
16.            if(key<la[mid])
17.                end=mid-1;
18.            else
19.                beg=mid+1;
20.        }
21.    }
22.    if(loc<0)
23.        printf("\n entered element doesn't exist into array!!!");
24.    getch();
25.    return 0;
26. }
```

[8]

2. Show all the steps of execution for the merge sorting on the below given array. Show the splitting and merging operation performed on the array. Clearly mention variable values during each step.

Array=[10, -3, 23, 41, 10, -1, 11, 5, 8,17, -2, 51, 101]

[10, -3, 23, 41, 10, -1, 11 || , 5, 8, 17, -2, 51, 101 -> -3, -2, 5, 8, 10, 17, 23, 41, 51, 101]

[10, -3, 23, 41 || 10, -1, 11 -> -3, 10, 23, 41] [5, 8, 17, || -2, 51, 101 -> -2, 5, 8, 17, 51, 101]

[10, -3 || 23, 41 -> -3, 10, 23, 41] [10, -1, || 11 -> -1, 10, 11] [5, 8, || 17 -> 5, 8, 17] [-2, 51, || 101 -> -2, 51, 101]

[10 || -3 -> -3, 10] [23 || 41 -> 23, 41] [10 || -1 -> -1, 10] [11 -> 11] [5 || 8 -> 5, 8] [17 -> 17] [-2 || 51 -> -2, 51] [101 -> 101]

[10 -> 10] [-3 -> 3] [23 -> 23] [41 -> 41] [10 -> 10] [-1 -> 1] [11 -> 11] [5 -> 5] [8 -> 8] [17 -> 17] [-2 -> -2] [51 -> 51] [101 -> 101]

[7]

3. Write the recurrence relation for Quick sort algorithm. Mention what is best, average and worst time complexity of Quick sort algorithm. [5]

Running time for partition of N elements is $\Theta(N)$ } Quicksort Running time: ° call partition. Get two subarrays of sizes NL and NR (what is the relationship between NL, NR, and N?) ° Then Quicksort the smaller parts ° $T(N) = N + T(NL) + T(NR)$

$$T(n) = T(k) + T(n-k-1) + \Theta(n)$$

BEST CASE RECURRENCE: $T(n) = 2T(n/2) + \Theta(n)$

BEST CASE TIME COMPLEXITY = $O(n \log n)$

AVERAGE CASE RECURRENCE: $T(n) = A(n) = 0$ if $n \leq 1$

SUM $k=1$ to n $((1/n)A(k-1) + A(n-k)) + (n-1)$ if $n \geq 2$

Average Case Complexity : $O(n \log n)$

Worst Case Recurrence: $T(n) = T(n) = T(0) + T(n-1) + \Theta(n)$

which is equivalent to

$$T(n) = T(n-1) + \Theta(n)$$

Worst Case Complexity = $O(n^2)$

4. Does Prim's and Kruskal's algorithms work if some edges have negative weights? Justify (prove or give counter example in each case). You may assume distinct edge weights. [5]

Yes it works for both Prim's and Kruskal's. Proof is by increasing the weights of all edges by the same constant C . (C is chosen such that after adding C to the weights of the edges the new edges have positive weight). Since the MST always has $n-1$ edges, the MST for the new graph after adding C will also be the MST for the original graph with negative edges.

5. For the following directed weighted tree, if the delay = 3, then list out the nodes that will be split according to the tree vertex splitting algorithm (which is a greedy algorithm). Also explain why these nodes will be the ones to be split. [5]

(i, c, g, d) will be split since their parent child edge sum exceeds delay of 3.

Sum for d = 4 > 3

Sum for c = 6 > 3

Sum for g = 5 > 3

Sum for i = 5 > 3

6. For the following example of a 0-1 knapsack problem show that the 0-1 knapsack problem does not have an optimal solution through the greedy approach. Evaluate the greedy solution value. Evaluate the optimal solution value. Show that the greedy solution value is less than the optimal solution value. [5]

0-1 Knapsack: Number of items = 3. Each item will either be selected or not selected in 0-1 knapsack.

Value[] = {60, 100, 120} ; Weight [] = {10, 20, 30}; $W = 50$

Sorted items: A (60/10), B (100/20), C (120/30)

Greedy approach = $A+B = 60 + 100 = 160$

Optimal solution = $B+C = 220$

Hence in this case of 0-1 knapsack greedy approach is less than the optimal solution.

7. Apply greedy algorithm on fractional knapsack problem to solve for the following case to find the optimal value packing of the knapsack. [5]

Example: Knapsack Capacity $W = 30$ and

Item	A	B	C	D
Value	50	140	60	60
Cost	5	20	10	12

Sorted items : A(50/5) B(140/20) C(60/10) D(60/12)

Greedy approach = $A + B + C/2 = 220$

Is the optimal solution

BITS PILANI, DUBAI CAMPUS
Dubai International Academic City, Dubai.
II Semester 2017-18

Design and Analysis of Algorithms (CS F364)

Date: 29.05.2018 Comp Exam

Max Marks: 80

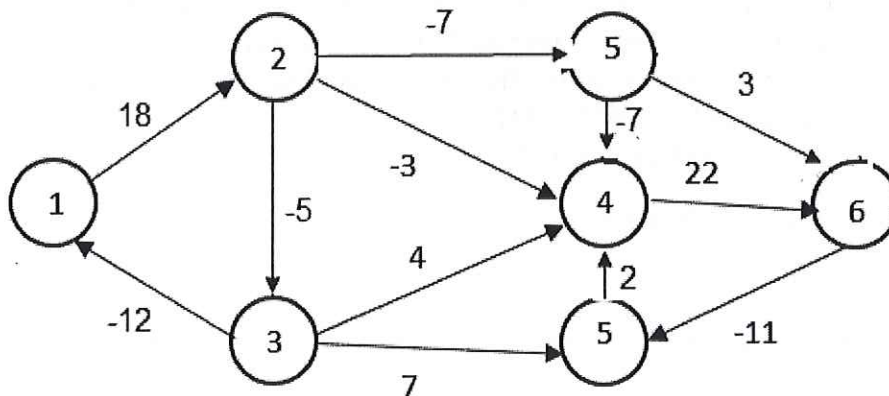
Time: 3 hours.

Weightage: 40%

Note: Write Part A and Part B in separate answer paper

PART A (4X10=40 marks)

1. We have given an array of integers having n elements. It is required to find minimum and maximum element in the array. Write divide and conquer algorithm for finding minimum and maximum. Derive the recursive relation for time complexity of algorithm to find minimum and maximum using divide and conquer.
2. Briefly explain working of Quick sort which uses divide and conquer method. What is worst time complexity for Quick sorting method? Give suitable example on at least 10 elements showing quick sort steps for worst time complexity.
3. Consider a knapsack with capacity $m = 44$, there are 6 objects whose weight are $(w_1, w_2, \dots, w_6) = (10, 6, 18, 9, 2, 11)$ and profit $(p_1, p_2, \dots, p_6) = (100, 20, 88, 90, 145, 51)$. With the fraction of items allowed in the knapsack, determine portion of objects $(0 \leq x \leq 1)$ has to be selected for each item to maximize the profit. Constraint is total weight of objects cannot exceed capacity of knapsack. Find maximum profit and portion of each object $(0 \leq x \leq 1)$ using Greedy method. Show all the steps clearly.
4. Determine single source all destination for the following graph. Write the matrix showing cost of single source all destination with source as node 1.



PART B (5*8=40 marks)

1. One of the two software packages, A or B, should be chosen to process data collections, containing each up to 109 records. Average processing time of the package A is $T_A(n) = 0.001 \times n$ milliseconds and the average processing time of the package B is $T_B(n) = 500 \times \sqrt{n}$ milliseconds. Which algorithm has better performance in a "Big-Oh" sense? Work out exact conditions when these packages outperform each other.

SOLUTION:

$T_A(n) = T_B(n)$ for $n = (5 \times 10^5)^2$. For n lesser, A performs better, For n greater B performs better. Asymptotically in the Big O notation B performs better.

2. Consider the following decision problems IS-COMPOSITE and IS-PRIME.

IS-COMPOSITE returns true if input number is composite, else it returns false.

IS-PRIME returns true if input number is prime, else it returns false.

COMPOSITE: A whole number that can be divided exactly by numbers other than 1 or itself.

Prime: A prime number is a whole number greater than 1 whose only factors are 1 and itself.

- a) Prove that IS-COMPOSITE reduces to IS-PRIME in polynomial time.

SOLUTION: Complement is a polynomial time operation.

- b) Prove that IS-COMPOSITE belongs to complexity class NP.

SOLUTION: Guess the factors, multiply them in polynomial time and verify.

- c) IS-PRIME whether it belongs to i) NP ii) NP-Hard iii) NP-Complete iv) P ? Justify your answer.

SOLUTION: IS-Prime belongs to Co-NP and P

- d) In 2002 Agrawal, Kayal and Saxena of IIT Kanpur (as part of Kayal and Saxena's BTech project) provided a deterministic algorithm that computes IS-PRIME(n) in time $O(\log^6 n)$. From this algorithm, what are the complexity classes of IS-PRIME and IS-COMPOSITE?

SOLUTION: both belong to P.

3. Questions on complexity classes:

- a) Prove that $P \subset NP$ and $P \subset coNP$.

[When the complement of a problem belongs to NP then the problem itself belongs to $coNP$]

SOLUTION: The class P is closed under complementation: if L is a language in P, then the complement of L is also in P. You can see this by taking any polynomial-time decider for L and switching the accept and reject states; this new machine now decides the complement of L and does so in polynomial time.

A language L is in $co-NP$ iff its complement is in NP. So consider any language $L \in P$. The complement of L is also in P, so the complement of L is therefore in NP (because $P \subseteq NP$). Therefore, L is in $co-NP$. Consequently, $P \subseteq co-NP$.

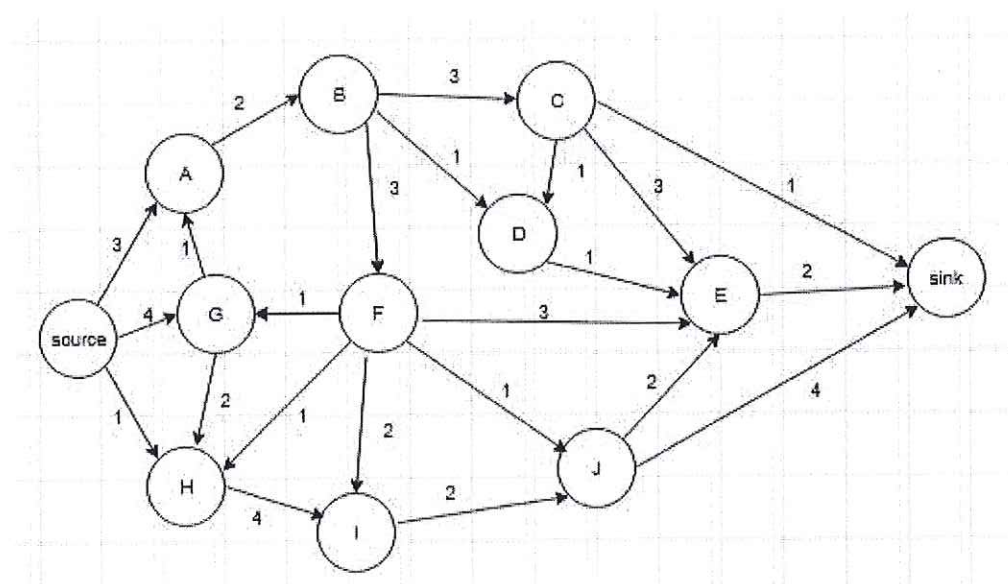
b) Show that, if $NP = P$ then $coNP = NP$. Is the reciprocal true?

SOLUTION: Because $P = coP$ would imply that $NP = coNP$.

c) Let $DOUBLE-SAT = \{\phi \mid \phi \text{ is a Boolean formula with two satisfying assignments}\}$. Show that $DOUBLE-SAT$ is NP-Complete. (Hint: Reduce 3SAT.)

SOLUTION: Reduction of 3SAT to $DOUBLE-SAT$: Given a 3cnf-function ψ , create a new Boolean function ψ_0 by adding a new clause $(x \vee x_{\text{bar}})$ to ψ , where x is a new variable not in ψ . Then check if $\psi_0 \in DOUBLE-SAT$. (3) This reduction clearly works in polynomial time.

4. Find out the max flow and min cut in the graph given below.



Max flow -4

Min cut = AB, GF, GH, sourceH

5. Give a polynomial time 2-approximation algorithm for the Vertex Cover problem. By 2-approximation algorithm we mean that the size of the vertex cover computed by our approximate algorithm is at most twice the most optimal vertex cover. Give proof for both the polynomial time complexity and for the ratio of 2 in the approximation.

A vertex cover of an undirected graph $G = (V, E)$ is a subset V^1 of the vertices of the graph which contains at least one of the two endpoints of each edge $V^1 \subseteq V : \forall \{a, b\} \in E, a \in V^1 \text{ OR } b \in V^1$ – The size of a vertex cover is the number of vertices in it.

SOLUTION:

1 $C \leftarrow \emptyset$

2 while $E \neq \emptyset$ pick any $\{u, v\} \in E$ $C \leftarrow C \cup \{u, v\}$ delete all edges incident to either u or v return C